Project 2: Particles Bouncing and Sticking on a Floor

Javad Yousefian, Alexandra VonHaven, Kevin Geumhan

October 21, 2026

Abstract

In this project we simulate particles that are shot from a source on the floor. They fly with gravity and linear air drag, and when they land they stick or they bounce. The start speeds come from the Maxwell–Boltzmann distribution (we use rejection sampling) and the directions come from the cosine law (we use the inverse CDF method). We move the particles with `solve_ivp`. We check the code with results that we can calculate by hand, we look at the errors from finite precision, numerical integration, the time step and the statistical sampling, and we measure how long the code needs. One time step needs $O(N)$ work and the full simulation needs $O(N/(p_{\text{stick}}\,\Delta t))$.

1 Introduction

In this project we simulate small particles, for example sand grains or droplets, that are shot from a small source on the floor. Every particle flies with gravity and air drag and then it lands on the floor. When it lands, it can stick to the floor or bounce again. We want to find where the particles stop in the end and how many times they bounce.

We chose this problem because it is simple, there is only one surface (the floor), and many results can be calculated by hand. So we can check our simulation in many ways. We wrote the simulation as a Python package with the name `bounce`, and we use numpy and scipy [1, 2].

In Section 2 we explain the model and in Section 3 the probability distributions. Section 4 is about the algorithm. In Section 5 we check the code, in Section 6 we look at the errors and in Section 7 at the computational complexity. The results for different levels of detail are in Section 8.

2 Model

2.1 Setup

The floor is the plane $z = 0$ and the gravity is in the $-z$ direction. All particles start at the origin. The particles do not interact with each other. We use units with $m = 1$ and $g = 1$.

2.2 Equation of motion

During the flight every particle follows Newton's second law with gravity and linear air drag:

$$m\frac{d^2\vec{r}}{dt^2} = -mg\,\hat{z} - \gamma\frac{d\vec{r}}{dt}, \tag{1}$$

where γ is the drag coefficient. We write it as two first order equations:

$$\frac{d\vec{r}}{dt} = \vec{v}, \qquad \frac{d\vec{v}}{dt} = -g\,\hat{z} - \frac{\gamma}{m}\,\vec{v}. \tag{2}$$

For this equation the exact solution is known, so we can use it to test the code. With $k = \gamma/m$ and $\vec{v}_t = (0, 0, g/k)$ it is

$$\vec{v}(t) = -\vec{v}_t + (\vec{v}_0 + \vec{v}_t)\, e^{-kt}, \tag{3}$$

$$\vec{r}(t) = \vec{r}_0 - \vec{v}_t\, t + (\vec{v}_0 + \vec{v}_t)\, \frac{1 - e^{-kt}}{k}. \tag{4}$$

After a long time a falling particle reaches the terminal velocity $v_t = mg/\gamma$. Without drag, a particle with speed v and angle θ from the vertical flies for the time $2v\cos\theta/g$ and it lands at the distance

$$R = \frac{v^2 \sin(2\theta)}{g}. \tag{5}$$

2.3 The floor

When a particle lands, it sticks with probability p_{stick}, and then it does not move anymore. If it does not stick, it bounces:

- With probability p_s the bounce is *specular*: v_z changes its sign and v_x, v_y stay the same, like a ball on a smooth floor.
- In the other case the bounce is *diffuse*: the particle keeps its speed but it gets a new random direction from the cosine law, like on a rough floor.

All bounces are elastic, so without drag the energy does not change. Every landing is independent, like a coin flip. So the number of bounces k follows

$$P(k) = (1 - p_{\text{stick}})^k\, p_{\text{stick}}, \qquad \langle k \rangle = \frac{1 - p_{\text{stick}}}{p_{\text{stick}}}. \tag{6}$$

3 Probability distributions

3.1 Speeds: rejection sampling

The particles come from a hot source, so we take the speeds from the Maxwell–Boltzmann distribution

$$f(v) = \sqrt{\frac{2}{\pi}}\, \frac{v^2}{\sigma^3}\, e^{-v^2/2\sigma^2}. \tag{7}$$

It is not easy to invert the CDF of this distribution, so we use rejection sampling [3]. We make a box around the curve, from $v = 0$ to $v = 5\sigma$ and from 0 to the maximum of the curve $f_{\max}$. The maximum is at $v = \sqrt{2}\,\sigma$. Then we take random points (v, y) in the box and we keep v when $y < f(v)$. The points that we keep are uniform under the curve, so the speeds that we keep follow $f(v)$. Above 5σ there are less than 2 particles in 10^5, so the box is wide enough.

The acceptance rate is the area under the curve divided by the area of the box, $1/(5\sigma f_{\max}) = 0.34$. With 10^5 speeds we got 0.340. The mean speed was $1.594\,\sigma$ (theory $2\sqrt{2/\pi}\,\sigma = 1.596\,\sigma$) and the mean of v^2 was $2.99\,\sigma^2$ (theory $3\,\sigma^2$). Figure 1 shows the histogram.

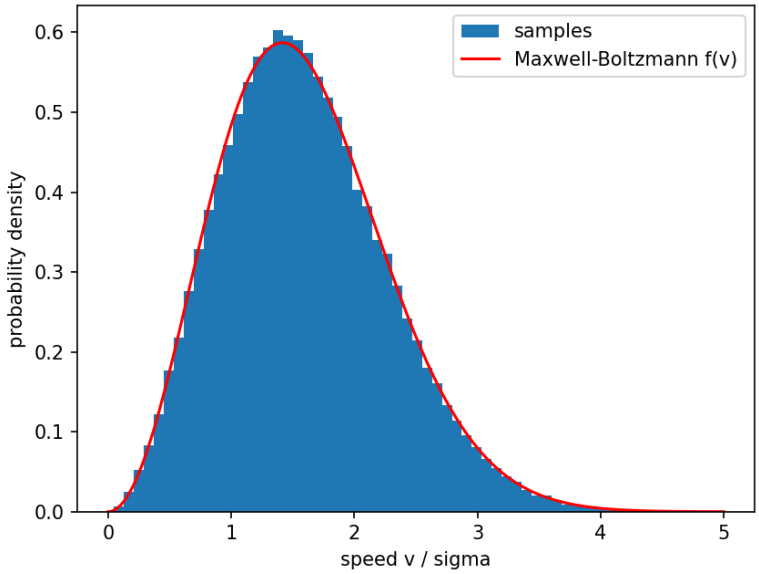


Figure 1: Histogram of 10^5 speeds and the Maxwell–Boltzmann distribution.

3.2 Directions: inverse CDF

The particles leave the source with the cosine law $p(\theta) \propto \cos\theta \sin\theta$, where θ is the angle from the vertical. After normalizing it is $p(\theta) = \sin 2\theta$, and the CDF is $\sin^2\theta$. When we set the CDF equal to a uniform random number u we get

$$\theta = \arcsin\sqrt{u}, \qquad \phi = 2\pi u', \tag{8}$$

where u' is a second uniform random number. We can take ϕ uniform because the source is the same in all directions. The direction is then $(\sin\theta\cos\phi, \sin\theta\sin\phi, \cos\theta)$. With this method no random numbers are thrown away. For the diffuse bounces we use the same law. Figure 2 shows the histogram of θ.

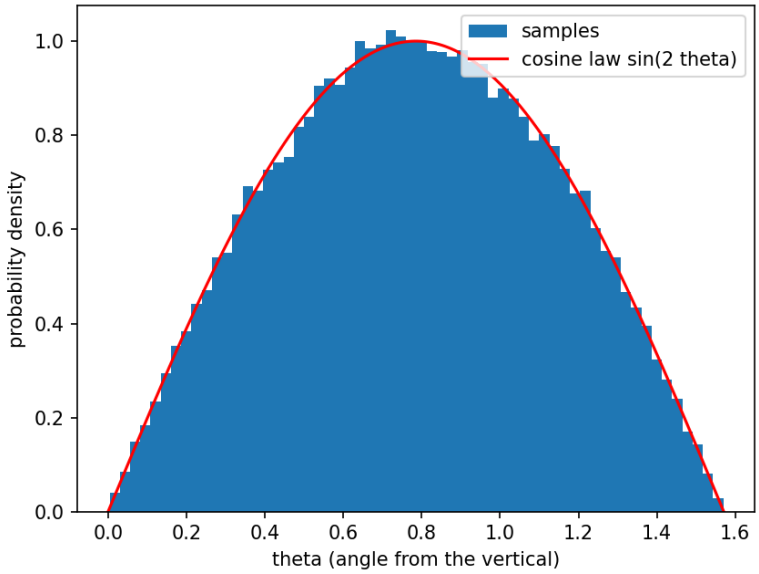


Figure 2: Histogram of 10^5 angles θ and the cosine law.

4 Algorithm

4.1 High-level algorithm

```
make N particles at the origin
    speeds:     Maxwell-Boltzmann (rejection sampling)
    directions: cosine law (inverse CDF)

while some particles are still flying:
    save old positions and velocities of the flying particles
    move all flying particles one time step dt      (solve_ivp)
    landed = flying particles with z < 0
    find landing point and velocity (linear interpolation)
    for each landed particle:
        stick with probability p_stick
        else specular bounce with probability p_s
        else diffuse bounce (new direction, same speed)

save final positions and number of bounces
```

4.2 Flight

We put the positions and velocities of all flying particles in one long array with length $6N$, and we move them one time step with `scipy.integrate.solve_ivp` (RK45) [2]. So one call moves all particles together. We give the solver the first step Δt, so it tries to do the whole step at once. Without drag the path is a parabola, and RK45 is exact for a parabola. With drag the solver makes smaller steps if the error is too big.

4.3 Landing

We only see that a particle landed at the end of a step, when it is already under the floor. It crossed $z = 0$ at the fraction $f = z_{\text{old}}/(z_{\text{old}} - z_{\text{new}})$ of the step. We find the landing point and the velocity at this moment with linear interpolation with the same f, and we put the particle on the floor.

4.4 Implementation

The code is a Python package and it can be installed with pip. The command `bounce run` runs one simulation, `bounce validate` runs all checks from Section 5 and the error studies, and `bounce benchmark` measures the time. We wrote tests with pytest for all parts of the code.

5 Validation

Here we compare the simulation with results that we can calculate by hand. Table 1 shows all checks with $\Delta t = 0.01$, $\sigma = 1$ and 10^4 particles. For a mean of N random values the error is $\text{std}/\sqrt{N}$, and we accept a difference up to 3 times this error. All checks pass.

Check	Result	Expected
first landing: max error$/(v_h \Delta t)$	0.11	≤ 1
mean first landing distance	2.363	2.356 ± 0.045
energy gained: max $\Delta E/E$	-3.2×10^{-8}	$\leq 10^{-12}$
energy loss/(landings $\cdot\, g^2 \Delta t^2/8$)	0.999999	≤ 1
mean final x	-0.028	0 ± 0.068
mean final y	0.022	0 ± 0.069
ϕ uniform (KS p-value)	0.57	≥ 0.01
random walk: mean $r^2/(k+1)$	9.95	10.0 ± 0.73
specular: mean $r/((k+1)R)$	0.99987	1 ± 0.001
specular: change of direction	1.6×10^{-13}	$\leq 10^{-9}$
terminal velocity: $\lvert v_z + v_t \rvert / v_t$	1.1×10^{-15}	$\leq 10^{-8}$
mean number of bounces	3.967	4.000 ± 0.133
$P(k)$: χ^2 p-value	0.039	≥ 0.003

Table 1: Validation checks. All of them pass.

Example 1: range. With $p_{\text{stick}} = 1$ and no drag every particle lands only once, so the landing must agree with Eq. (5). We only see the landing at the end of a step, so the error can not be bigger than the distance of one step, $v_h \Delta t$. For the worst particle the error is 0.11 of this. The speed and the direction are independent, so the mean range is $\langle v^2 \rangle \langle \sin 2\theta \rangle / g = 3\sigma^2 \cdot \frac{\pi}{4} / g = 2.356$. We get 2.363 ± 0.015.

Example 2: energy. Without drag the energy can only change at a landing, because of the interpolation. The straight line between two points is under the parabola, so at the interpolated time the particle is really a little bit above the floor, at most $g\Delta t^2/8$. This means the energy can only go down, and at most by $mg^2\Delta t^2/8$ at every landing. In the simulation the energy loss stays under this limit.

This check found a mistake in our first version. There we used the velocity from the end of the step. But at this time the particle was already under the floor and too fast, so it got more energy

at every bounce, and some particles got more than 100% more energy. After we also interpolated the velocity, no particle gets energy anymore.

Example 3: random walk. With only diffuse bounces every flight goes in a new random direction, so the flights are independent and their mean is zero. Then $\langle r^2 \rangle = (k+1)\langle R^2 \rangle$ after k bounces, with $\langle R^2 \rangle = \langle v^4 \rangle \langle \sin^2 2\theta \rangle / g^2 = 15\sigma^4 \cdot \frac{2}{3}/g^2 = 10\sigma^4/g^2$. We get 9.95 ± 0.24.

Example 4: number of bounces. Every landing is independent, so the number of bounces must follow $P(k)$ from Eq. (6). With 10^4 particles the mean is 3.97 ± 0.04, and the theory gives $(1 - 0.2)/0.2 = 4$. We also compared the whole histogram with $P(k)$ with a χ^2 test. The p-value is 0.039, so the simulation agrees with $P(k)$.

Other checks. The source is the same in all directions, so the mean x and y must be zero and the angle ϕ must be uniform. We test this with drag ($\gamma = 0.2$) and with the Kolmogorov–Smirnov test. With only specular bounces all flights of a particle are the same, so it must stop at $r = (k+1)R$ in the direction where it started. With drag $\gamma = 0.5$, a particle that falls for a long time must reach $v_z = -mg/\gamma$.

6 Errors

6.1 Finite precision

Without drag RK45 is exact, so in a flight without the floor the energy can only change because of round-off. We fly the same time $t = 2$ with more and more steps (Table 2). The error gets bigger with more steps, but it stays under 10^{-10}, so it is not important for our results.

steps	Δt	max $\lvert\Delta E\rvert/E$
10	0.2	2.5×10^{-14}
100	0.02	1.4×10^{-13}
1000	0.002	1.0×10^{-13}
10000	0.0002	1.3×10^{-11}

Table 2: Energy error from round-off in a flight without drag.

6.2 Numerical integration

We compare `solve_ivp` with the exact solution with drag ($\gamma = 0.5$, time 5, `atol` = `rtol`/1000). When we call it only once for the whole time, the solver chooses its own steps and the error follows `rtol`. In the simulation we call it once for every $\Delta t = 0.01$, and then the error is about 10^{-14} for every tolerance (Table 3).

`rtol`	error (one call)	error (steps of Δt)
10^{-3}	3.7×10^{-4}	1.2×10^{-14}
10^{-6}	3.9×10^{-7}	1.2×10^{-14}
10^{-10}	2.7×10^{-11}	1.2×10^{-14}

Table 3: Maximum position error of `solve_ivp` with drag.

6.3 Time step

The interpolation of the landing gives an error that depends on Δt. If the particle lands at the time s inside the step, the interpolated point is a bit too early, and there the particle is still $gs(\Delta t - s)/2$ above the floor. The landing time can be anywhere in the step with the same chance, and the mean of $s(\Delta t - s)$ is $\Delta t^2/6$. With $\langle \tan\theta \rangle = \pi/2$ we get

$$\langle \Delta E \rangle = \frac{mg^2\Delta t^2}{12}, \qquad \langle |\Delta R| \rangle = \frac{\pi g \Delta t^2}{24}. \tag{9}$$

In Table 4 we see that the simulation agrees with this. A fit gives the order $\Delta t^{2.00}$ for both errors.

Δt	mean $\lvert\Delta R\rvert$	$\pi g\Delta t^2/24$	mean ΔE	$g^2\Delta t^2/12$
0.04	2.03×10^{-4}	2.09×10^{-4}	1.33×10^{-4}	1.33×10^{-4}
0.02	5.06×10^{-5}	5.24×10^{-5}	3.34×10^{-5}	3.33×10^{-5}
0.01	1.27×10^{-5}	1.31×10^{-5}	8.33×10^{-6}	8.33×10^{-6}
0.005	3.16×10^{-6}	3.27×10^{-6}	2.06×10^{-6}	2.08×10^{-6}
0.0025	7.95×10^{-7}	8.18×10^{-7}	5.20×10^{-7}	5.21×10^{-7}

Table 4: Time step errors from the simulation (10^4 particles) and from the theory.

6.4 Statistical sampling

Every result is a mean over N random particles, so the error is $\text{std}/\sqrt{N}$. For the mean final distance we get 0.30, 0.082 and 0.027 for $N = 10^3$, 10^4 and 10^5 (Table 7). A fit on a log-log plot gives the slope -0.53, which is close to $-1/2$ (Figure 3). For the mean first landing distance we know the exact value $3\pi\sigma^2/(4g)$, so here we can also see the real error. It is 0.083, 0.0044 and 0.0014 for the three values of N. So it goes down in the same way, and it has the same size as $\text{std}/\sqrt{N}$.

Summary. With $\Delta t = 0.01$ the time step error is about 10^{-5}, the integration error is about 10^{-14} and the round-off is under 10^{-10}. The statistical error is about 10^{-2}. So our accuracy is limited by the number of particles and not by the numerics.

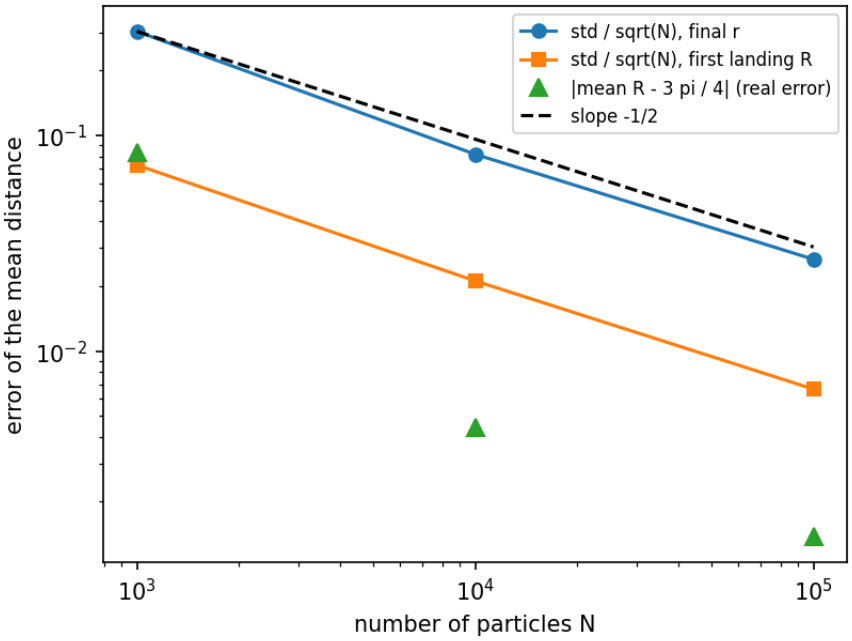


Figure 3: Statistical error std/$\sqrt{N}$ of the mean final distance and of the mean first landing distance, and the real error $|\langle R\rangle - 3\pi/4|$ of the first landing, for different numbers of particles.

7 Computational complexity

7.1 From first principles

In one time step we move all N_{fly} flying particles. One call of RK45 needs 7 evaluations of the right-hand side, and each of them is a vector operation on $6N_{\text{fly}}$ numbers. The floor only looks at the particles that landed. So one step needs $O(N_{\text{fly}})$ work, plus a fixed cost for calling `solve_ivp`.

The total work is the sum of N_{fly} over all steps. Every particle has $(k+1)$ flights, and one flight takes on average $\langle 2v\cos\theta/g\rangle = 2\langle v\rangle\langle\cos\theta\rangle/g = 2\cdot 1.596\,\sigma\cdot\frac{2}{3}/g = 2.13\,\sigma/g$. So the number of particle-steps for one particle is

$$\frac{\langle k+1\rangle\cdot 2.13\,\sigma/g}{\Delta t} = \frac{2.13\,\sigma/g}{p_{\text{stick}}\,\Delta t}, \tag{10}$$

and the total work is $O(N/(p_{\text{stick}}\,\Delta t))$. For $p_{\text{stick}} = 0.2$ and $\Delta t = 0.01$ this gives 1064 steps per particle. The number of time steps depends on the particle that stays in the air for the longest time. The largest k of N particles grows like $\ln N/\ln(1/(1-p_{\text{stick}}))$, so the number of steps grows only like $\ln N$. The memory is $O(N)$.

7.2 Numerical tests

Table 5 shows the time of one step when all particles are flying. A fit gives time $\propto N^{1.04}$, so it is linear like we expected. The measured particle-steps per particle (Table 7) are 1156, 1057 and 1069, which is close to the 1064 that we predicted. For different Δt (Table 8) they are 272, 1057 and 4276, so they go like $1/\Delta t$ like we predicted (266, 1064, 4256).

N	time per step (s)	time per particle (s)
1000	1.7×10^{-4}	1.7×10^{-7}
10000	1.1×10^{-3}	1.1×10^{-7}
100000	2.1×10^{-2}	2.1×10^{-7}

Table 5: Time of one step when all particles are flying.

7.3 Benchmarks and profiling

A full run needed 1.6 s for $N = 10^3$, 5.6 s for $N = 10^4$ and 36 s for $N = 10^5$, so the total time grows like $N^{0.68}$. This is slower than N, because in most steps only a few particles are still flying and every step has a fixed cost for calling `solve_ivp`. The time per particle-step goes down from 1.4×10^{-6} s to 5.3×10^{-7} s and 3.4×10^{-7} s. It becomes almost constant only for large N, where the work for the particles is the biggest part.

We used `cProfile` for a run with $N = 10^4$, which needed 5.4 s (Table 6). The slowest part is `solve_ivp`, like we thought in our plan. Most of this time is the work of the solver itself (the Runge–Kutta step), and not our physics: our right-hand side needs only 7%. Counting the flying particles in every step (`n_flying`) also needs 7%, which is more than we thought. When we use the exact solution instead of `solve_ivp`, the same run is about three times faster.

Function	part of the time
`move` (all of the flight)	74%
`solve_ivp`	62%
Runge–Kutta step	47%
our right-hand side	7%
`n_flying` (count flying particles)	7%
`land` (floor)	$< 4\%$

Table 6: Profile of a run with $N = 10^4$ (cumulative time).

With the benchmark we also found a memory problem: for $N = 10^5$ the memory went up to several GB, because the old solver objects were not deleted fast enough. We fixed it by cleaning the memory after every call of `solve_ivp`. Now the run needs about 170 MB, like we expect for $O(N)$.

8 Results

If we do not say something else, we use $p_{\text{stick}} = 0.2$, $p_s = 0.5$, $\sigma = 1$ and no drag.

8.1 Levels of detail: number of particles

Table 7 shows the results for $N = 10^3$, 10^4 and 10^5. The mean distance and the mean number of bounces are more precise with more particles, and the error goes down like $1/\sqrt{N}$. With 10^5 particles the mean number of bounces is 4.007 ± 0.014, very close to the theory $(1 - 0.2)/0.2 = 4$. With 10^3 particles it is 4.41 ± 0.16, which is 2.6 errors away. So with only a few particles the result can be quite far from the theory just by chance. The largest number of bounces gets bigger with N, because with more particles there are more rare particles that bounce many times. The mean first landing distance also gets closer to the theory $3\pi\sigma^2/(4g) = 2.356$ with more particles: we get 2.44 ± 0.07, 2.361 ± 0.021 and 2.355 ± 0.007 for $N = 10^3$, 10^4 and 10^5.

N	mean r	mean k	max k	steps	particle-steps/N
1000	7.23 ± 0.30	4.41 ± 0.16	43	13799	1156
10000	6.54 ± 0.08	3.97 ± 0.04	43	10923	1057
100000	6.59 ± 0.03	4.007 ± 0.014	62	24039	1069

Table 7: Results for three numbers of particles ($\Delta t = 0.01$).

8.2 Levels of detail: time step

Table 8 shows the results for three time steps with $N = 10^4$. The mean distance changes less than its statistical error, because the time step error ($\sim \Delta t^2$) is much smaller. A smaller time step only makes the run slower: the number of particle-steps grows like $1/\Delta t$.

Δt	mean r	mean k	particle-steps/N
0.04	6.63 ± 0.09	4.06	272
0.01	6.54 ± 0.08	3.97	1057
0.0025	6.66 ± 0.08	4.02	4276

Table 8: Results for three time steps ($N = 10^4$).

8.3 Final positions and bounces

Figure 4 shows where the particles stopped in one run with $N = 10^4$. They make a round cloud around the source, because no direction is special. Most particles stop close to the source and only a few go very far (Figure 5). The mean final distance is 6.79, and the mean first landing is 2.365, which is close to the theory 2.356. The number of bounces follows $P(k)$, and on a log scale this is a straight line (Figure 6). The mean number of bounces is 4.03 and the theory is 4.

8.4 Specular, diffuse and drag

In Figure 7 we compare specular and diffuse bounces, with and without drag ($N = 10^4$). With specular bounces and no drag a particle keeps its direction, so it goes far: the mean distance is 12.1. With diffuse bounces the particle does a random walk and it stays closer to the source: 4.5. Drag makes all flights shorter: 2.6 for specular and 1.7 for diffuse with $\gamma = 0.3$.

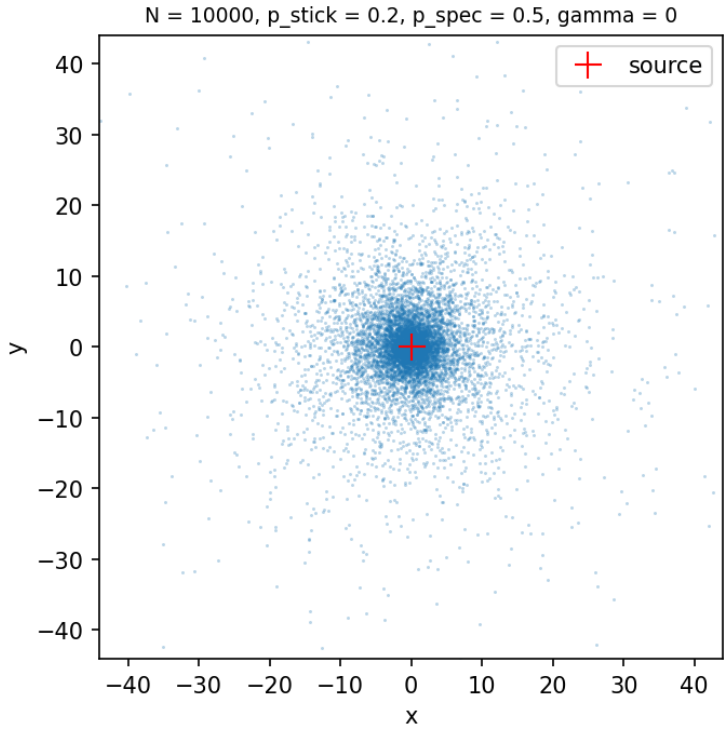


Figure 4: Final positions of the particles on the floor ($N = 10^4$).

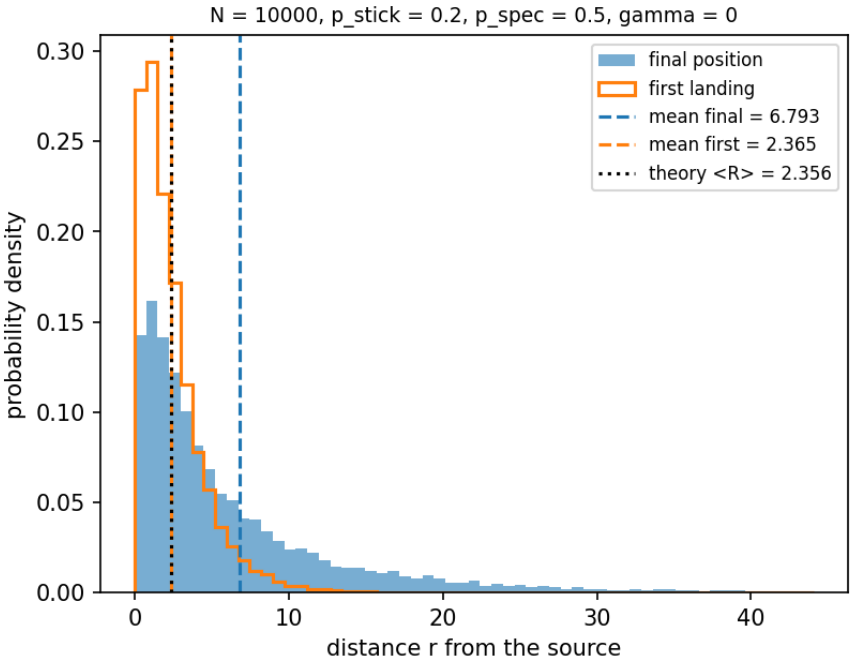


Figure 5: Distribution of the final distance r and of the first landing distance, with the theory $\langle R \rangle = 3\pi\sigma^2/(4g)$ ($N = 10^4$).

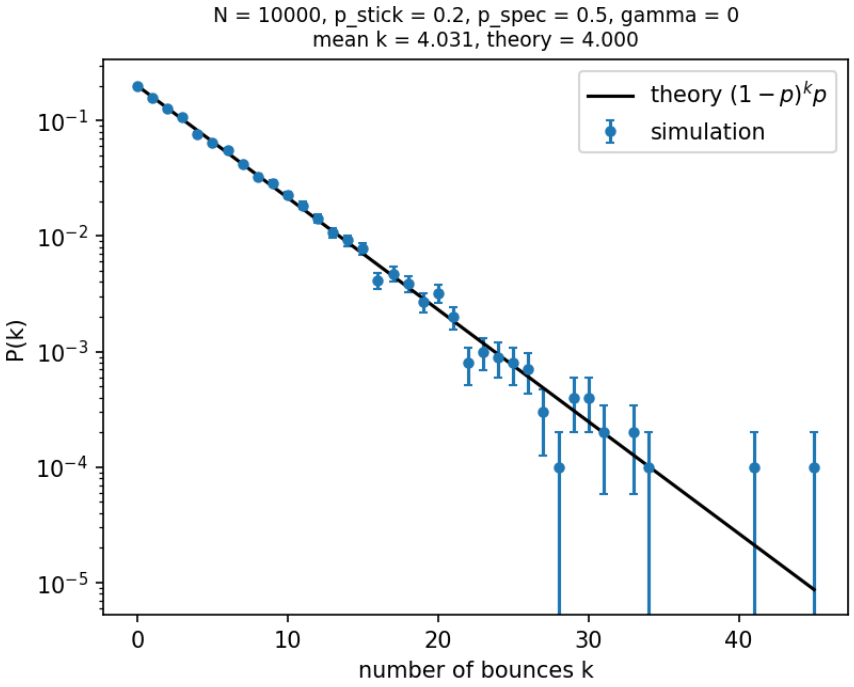


Figure 6: Number of bounces and the theory $P(k)$ from Eq. (6) ($N = 10^4$). The error bars are $\sqrt{\text{count}}/N$.

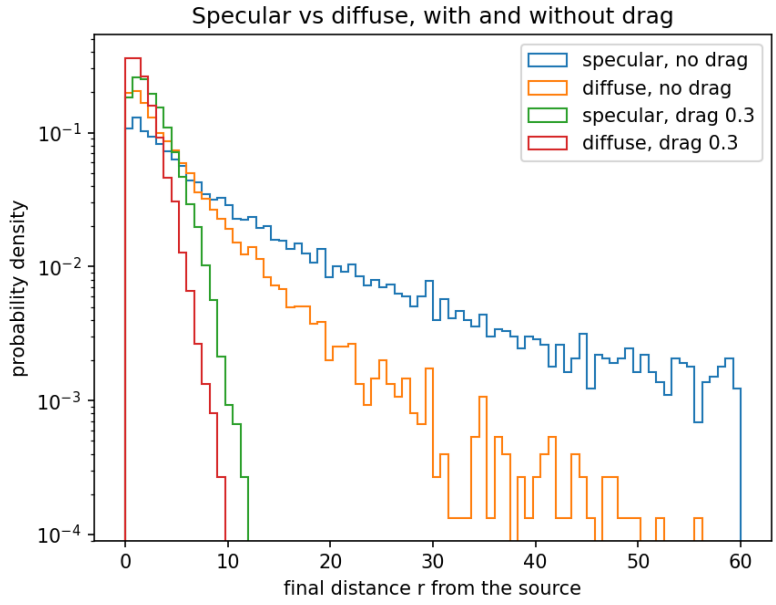


Figure 7: Final distance for specular and diffuse bounces, with and without drag.

9 Conclusion

We simulated particles that fly with gravity and air drag and then stick or bounce on a floor. All our validation checks pass, and the energy check also helped us to find a mistake in the landing. The biggest numerical error comes from the time step and goes like Δt^2, but it is much smaller than the statistical error, which goes like $1/\sqrt{N}$. One time step needs $O(N)$ work and the whole run needs $O(N/(p_{\mathrm{stick}}\Delta t))$, and our measurements agree with this. Most of the time is spent in `solve_ivp`. With specular bounces the particles go much farther than with diffuse bounces, and drag makes all distances shorter.

References

[1] C. R. Harris et al., *Array programming with NumPy*, Nature **585**, 357–362 (2020).

[2] P. Virtanen et al., *SciPy 1.0: fundamental algorithms for scientific computing in Python*, Nature Methods **17**, 261–272 (2020).

[3] W. H. Press, S. A. Teukolsky, W. T. Vetterling and B. P. Flannery, *Numerical Recipes*, 3rd ed., Cambridge University Press (2007).